# Сведения о файле

**Цель работы**

Научиться получать безопасно доступные сведения о тестовом файле в Python: проверять существование файла, отличать файл от отсутствующего объекта, выводить имя, расширение, полный путь, родительскую папку, размер и время последнего изменения. Научиться корректно обрабатывать ситуацию, когда указанный файл не существует.

*Среда выполнения: Python 3.x, Visual Studio Community. Все действия выполняются только с тестовыми файлами в рабочей папке проекта. Продолжительность — 2 академических часа.*

## 1. Связь с предыдущей практической работой

В предыдущей работе программа создавала, читала и дополняла текстовые файлы. Теперь содержимое файла не является главной темой. Нужно научиться получать сведения о самом файле: как он называется, где находится, сколько занимает места и существует ли он вообще.

Для работы снова используется pathlib и объект Path. Это позволяет выполнять все действия внутри пользовательской рабочей папки и не обращаться к системным каталогам Windows.

**Граница темы**

В этой работе не создаются и не удаляются каталоги, не выполняются копирование, перемещение и переименование файлов, не изучаются кодировки и обработка исключений try/except. Эти действия относятся к следующим темам рабочей программы.

## 2. Подготовка тестового файла

Чтобы у всех был предсказуемый объект для проверки, сначала создадим небольшой тестовый файл в рабочей папке проекта. Запись текста уже изучалась ранее, поэтому здесь этот фрагмент используется только для подготовки данных.

**Подготовка файла sample_info.txt**

```
from pathlib import Path

file_path = Path.cwd() / "sample_info.txt"

with open(file_path, "w", encoding="utf-8") as file:
    file.write("Системное программирование\n")
    file.write("Тестовый файл для изучения свойств\n")

print("Тестовый файл подготовлен:", file_path)
```

После запуска в рабочей папке должен появиться sample_info.txt. Откройте его обычным способом и убедитесь, что файл существует.

**ЗАДАНИЕ 1. Подготовить собственный тестовый файл**

Создайте проект Python с именем FileInfoPractice или откройте отдельную копию учебного проекта.
Создайте в рабочей папке файл student_data.txt программным способом.
Запишите в него минимум две короткие строки по своему выбору.
После создания выведите полный путь к файлу.
Сделайте скриншот консоли для отчёта и подпишите, какой файл был создан.

## 3. Что хранит объект Path

Объект Path содержит путь к файлу и позволяет получить отдельные части этого пути без ручной обработки строки.

**Базовый пример Path**

```
from pathlib import Path

file_path = Path.cwd() / "sample_info.txt"

print("Объект Path:", file_path)
print("Имя файла:", file_path.name)
print("Расширение:", file_path.suffix)
print("Родительская папка:", file_path.parent)
```

**Пример результата**

```
Объект Path: C:\Users\Student\source\repos\FileInfoPractice\sample_info.txt
Имя файла: sample_info.txt
Расширение: .txt
Родительская папка: C:\Users\Student\source\repos\FileInfoPractice
```

На разных компьютерах родительская папка будет отличаться. Имя файла и расширение при этом останутся такими, какие заданы в программе.

| Свойство / метод | Что возвращает | Пример для sample_info.txt |
|---|---|---|
| .name | Имя файла вместе с расширением. | sample_info.txt |
| .suffix | Расширение файла. | .txt |
| .parent | Родительскую папку. | ...\FileInfoPractice |
| .resolve() | Полный путь к файлу. | C:\...\sample_info.txt |
| .exists() | True, если путь существует. | True / False |
| .is_file() | True, если путь указывает на файл. | True / False |

## 4. Проверка существования файла

Перед получением размера и других свойств полезно сначала проверить, существует ли указанный путь. Для этого используется метод exists().

**Проверка exists()**

```
from pathlib import Path

file_path = Path.cwd() / "sample_info.txt"

if file_path.exists():
    print("Файл найден.")
else:
    print("Файл не найден.")
```

Метод exists() возвращает логическое значение True или False. Поэтому его удобно использовать непосредственно в условии if.

### 4.1. Проверка, что путь указывает именно на файл

Путь может существовать, но указывать не на обычный файл. Для дополнительной проверки используется is_file(). В этой практической мы работаем только с файлами.

**Проверка exists() и is_file()**

```
if file_path.exists() and file_path.is_file():
    print("Путь существует и указывает на файл.")
else:
    print("Нужный файл не найден.")
```

**Важно**

Не подставляйте в программу пути к системным файлам Windows. Для всех заданий используйте только файлы, созданные вами в рабочей папке проекта.

**ЗАДАНИЕ 2. Проверить существующий и отсутствующий файл**

Проверьте student_data.txt с помощью exists() и is_file().
Затем измените имя в коде на missing_data.txt — такой файл специально не создавайте.
Запустите программу повторно.
В отчёте разместите два результата и кратко объясните, почему во втором случае программа выводит сообщение об отсутствии файла.

## 5. Полный путь к файлу

Иногда требуется показать не относительный, а полный путь. Для этого можно использовать resolve().

**Получение полного пути**

```
full_path = file_path.resolve()

print("Полный путь:", full_path)
```

Метод resolve() удобен для отчёта и диагностики: он показывает, с каким конкретно файлом работает программа.

**Не копируйте путь из примера**

Всегда используйте путь, который сформировала ваша программа. Папка пользователя, имя проекта и расположение репозитория могут отличаться.

## 6. Размер файла

Сведения файловой системы можно получить методом stat(). Он возвращает набор характеристик файла. В этой работе нас в первую очередь интересует st_size — размер файла в байтах.

**Получение размера файла**

```
info = file_path.stat()

print("Размер файла:", info.st_size, "байт")
```

Байт — базовая единица объёма данных. Для небольшого текстового файла размер обычно составляет десятки или сотни байтов. Точное значение зависит от текста и кодировки.

### 6.1. Почему сначала нужна проверка exists()

Если вызвать stat() для несуществующего файла, программа завершится ошибкой. Поэтому сначала проверяем наличие файла, а только затем читаем его свойства.

**Безопасная последовательность**

```
if file_path.exists() and file_path.is_file():
    info = file_path.stat()
    print("Размер:", info.st_size, "байт")
else:
    print("Файл не найден.")
```

**ЗАДАНИЕ 3. Получить размер собственного файла**

Для student_data.txt выведите размер в байтах.
Перед вызовом stat() обязательно проверьте exists() и is_file().
Запишите значение размера в отчёт.
Добавьте в student_data.txt ещё одну строку известным из прошлой работы способом и снова запустите проверку размера.
Сравните два значения и своими словами объясните, почему размер изменился.

## 7. Время последнего изменения

Объект stat() содержит также st_mtime — время последнего изменения файла. Оно хранится как числовая временная метка. Чтобы получить понятную дату и время, используем модуль datetime.

**Преобразование времени изменения**

```
from datetime import datetime

info = file_path.stat()
modified_time = datetime.fromtimestamp(info.st_mtime)

print("Изменён:", modified_time.strftime("%d.%m.%Y %H:%M:%S"))
```

Метод fromtimestamp() преобразует числовую временную метку в дату и время, а strftime() задаёт удобный формат вывода.

**Что считать изменением**

Если файл открыть и сохранить с новым содержимым, время последнего изменения обновится. Простое чтение файла обычно не меняет это значение.

**ЗАДАНИЕ 4. Зафиксировать время изменения**

Выведите время последнего изменения student_data.txt.
После этого добавьте в файл ещё одну короткую строку, сохраните его и повторно запустите программу.
Сравните два значения времени.
В отчёте поясните, какое действие привело к изменению даты/времени.

## 8. Собираем сведения в одной программе

Теперь объединим основные действия. Сначала программа формирует путь, затем проверяет его, и только после успешной проверки выводит свойства.

**Полный учебный пример**

```
from pathlib import Path
from datetime import datetime

file_path = Path.cwd() / "sample_info.txt"

if file_path.exists() and file_path.is_file():
    info = file_path.stat()

    print("Файл найден")
    print("Имя:", file_path.name)
    print("Расширение:", file_path.suffix)
    print("Полный путь:", file_path.resolve())
    print("Папка:", file_path.parent)
    print("Размер:", info.st_size, "байт")

    modified = datetime.fromtimestamp(info.st_mtime)
    print("Изменён:", modified.strftime("%d.%m.%Y %H:%M:%S"))
else:
    print("Файл не найден:", file_path.name)
```

**Пример результата**

```
Файл найден
Имя: sample_info.txt
Расширение: .txt
Полный путь: C:\Users\Student\source\repos\FileInfoPractice\sample_info.txt
Папка: C:\Users\Student\source\repos\FileInfoPractice
Размер: 86 байт
Изменён: 28.09.2026 18:40:12
```

Числа и путь в вашем результате будут другими. Это нормально: программа должна показывать фактические свойства именно вашего файла.

## 9. Почему порядок действий важен

Правильный алгоритм этой практической можно описать так:

| Шаг | Действие |
|---|---|
| 1 | Сформировать путь к тестовому файлу в рабочей папке. |
| 2 | Проверить exists() и is_file(). |
| 3 | Если файл найден — получить stat(). |
| 4 | Вывести имя, расширение, путь и родительскую папку. |
| 5 | Вывести размер и время последнего изменения. |
| 6 | Если файла нет — вывести понятное сообщение и не обращаться к stat(). |

Такой порядок нужен потому, что часть свойств можно безопасно получать только после подтверждения, что файл действительно существует.

## 10. Итоговая самостоятельная работа

Итоговая программа должна быть похожа по логике на учебный пример, но использовать другой файл и самостоятельно организованный вывод.

**ЗАДАНИЕ 5. Программа «Инспектор файла»**

Создайте программу «Инспектор файла».

Требования:

1. В рабочей папке создайте тестовый файл report.txt и запишите в него минимум три строки.
2. Программа должна сформировать путь к report.txt через Path.cwd().
3. Сначала проверить exists() и is_file().
4. Если файл найден, вывести:
   - имя файла;
   - расширение;
   - полный путь;
   - родительскую папку;
   - размер в байтах;
   - время последнего изменения.
5. Если файл отсутствует, вывести понятное сообщение и не вызывать stat().
6. После успешного запуска измените имя в программе на report_missing.txt и покажите корректную реакцию на отсутствующий файл.
7. Не использовать системные каталоги, удаление, переименование, копирование файлов или try/except.

## 11. Дополнительное самостоятельное изменение

**ЗАДАНИЕ 6. Добавить одну полезную характеристику**

Выберите один вариант и добавьте его в программу:

- вывести имя файла без расширения через stem;
- вывести размер одновременно в байтах и приблизительно в килобайтах;
- вывести отдельное сообщение, если размер файла равен 0 байт.

Выбранное изменение кратко объясните в отчёте. Готовый код для этого пункта в практической не приводится.

## 12. Если результат отличается от ожидаемого

| Проблема | Что проверить |
|---|---|
| Файл не находится | Выведите Path.cwd() и убедитесь, что файл создан именно в этой рабочей папке. |
| exists() возвращает False | Проверьте имя и расширение файла, включая .txt. |
| stat() вызывает ошибку | Не вызывайте stat(), пока exists() и is_file() не подтвердили наличие файла. |
| Размер кажется неожиданным | Размер измеряется в байтах и зависит от фактического содержимого и кодировки. |
| Время изменения не поменялось | Убедитесь, что файл действительно был сохранён после изменения содержимого. |
| Путь отличается от примера | Это нормально: рабочая папка зависит от компьютера и расположения проекта. |

## 13. Что должно быть в Word-отчёте

Основной сдаваемый результат — Word-документ. Один исходный проект без отчёта не считается полностью оформленной практической работой.

| Раздел отчёта | Что должно быть в Word |
|---|---|
| Тема и цель | Название практической работы и цель. |
| Подготовка файла | Скриншот созданного student_data.txt или консоли с путём к нему + пояснение. |
| Проверка существования | Результат для существующего и отсутствующего файла + пояснение. |
| Свойства файла | Скриншот вывода имени, расширения, пути, размера и времени изменения. |
| Итоговая программа | Полный код программы «Инспектор файла» текстом с сохранением отступов. |
| Итоговый запуск | Скриншот успешного анализа report.txt и отдельный результат для report_missing.txt. |
| Таблица наблюдений | Заполненная таблица из раздела 14. |
| Контрольные вопросы | Ответы на 10 вопросов из раздела 15. |
| Вывод | 3–5 предложений: какие свойства файла получены и как обработано отсутствие файла. |

**Правило для скриншотов**

Каждый скриншот должен иметь подпись «Рисунок N — ...». Под скриншотом студент пишет 1–3 предложения своими словами: что показано, какое действие выполнялось и какой результат подтверждает изображение.

Исходный код итоговой программы вставляется в Word текстом с сохранением отступов. Скриншот редактора не заменяет текст кода.

## 14. Таблица наблюдений

Заполните таблицу по фактическим данным вашего файла. Не копируйте значения из примеров.

| Характеристика | Фактическое значение | Краткое пояснение |
|---|---|---|
| Имя файла | | |
| Расширение | | |
| Полный путь | | |
| Родительская папка | | |
| Размер до изменения, байт | | |

| | | |
|---|---|---|
| Размер после изменения, байт | | |
| Время последнего изменения | | |
| Результат проверки отсутствующего файла | | |

## 15. Вопросы для проверки понимания

1. Для чего используется объект Path?
2. Что возвращает file_path.exists()?
3. Для чего нужен метод is_file()?
4. Чем file_path.name отличается от file_path.suffix?
5. Для чего используется resolve()?
6. Что показывает st_size?
7. Почему stat() не следует вызывать до проверки существования файла?
8. Что означает st_mtime?
9. Почему в этой работе используются только файлы из рабочей папки проекта?
10. Как программа должна вести себя, если указанного файла нет?

## 16. Результат выполнения

- создан безопасный тестовый файл в рабочей папке проекта;
- получены имя, расширение, полный путь и родительская папка файла;
- проверены exists() и is_file();
- получен размер файла через stat().st_size;
- получено время последнего изменения через stat().st_mtime;
- корректно обработана ситуация отсутствия файла без вызова stat();
- создана итоговая программа «Инспектор файла»;
- оформлен Word-отчёт со скриншотами, кодом, наблюдениями, ответами и выводом.

**Критерий готовности**

Работа завершена, если программа запускается без синтаксических ошибок, анализирует только тестовый файл в рабочей папке, выводит требуемые свойства существующего файла, корректно сообщает об отсутствии файла и не выполняет запрещённых операций с системными каталогами.